

EMG Band

Two-channel myoelectric band for Kyle. Drives the Brunel Hand, the Robo Kyle arcade, and any Bluetooth device. Final spec v2.1, September 2026.

1. What it is

A sleeve worn on the forearm that reads how hard two muscles are working and which way the arm is pointing, and sends that out as a stream of numbers. The band never receives commands. It has three modes: streaming to the Factum server, driving the hand directly with no server, and acting as a Bluetooth mouse. The band and the hand are separate devices with their own batteries; they never share a wire.

```
KYLE'S FOREARM
|
+-- SLEEVE ..... compression elbow sleeve, snug, below the elbow crease
|   +-- SENSOR 1 ..... flexor pad, inside of forearm, top third  -> CLOSE
|   +-- SENSOR 2 ..... extensor pad, outside of forearm, top third -> OPEN
|   +-- CINCH STRAP .. 2 in elastic over the pads, adjustable pressure
|
+-- POD ..... printed box on the outer side of the sleeve
    +-- Pico 2 W ..... brain, Wi-Fi, Bluetooth
    +-- BN0085 ..... arm orientation (yaw, pitch, roll)
    +-- LiPo + SHIM .. battery, charging, power switch
    +-- button ..... tap = recenter, 1 s = mode, 2 s = ESTOP
    +-- motor ..... haptic click
    |
    +-- MODE 1  HAND-FACTUM  Wi-Fi  -> UDP frames -> Factum -> POST -> hand
    +-- MODE 2  HAND-DIRECT  own hotspot -> band classifies -> POST -> hand
    +-- MODE 3  MOUSE        Bluetooth HID -> any TV, phone, laptop
```

2. How the signal works

When a nerve fires, every muscle fiber it controls produces a small electrical pulse as it contracts. Thousands of pulses overlap and reach the skin as a fuzzy signal about a thousandth of a volt, swinging positive and negative a few hundred times a second. The harder the muscle works, the more fibers fire and the louder the fuzz. The band measures how loud that fuzz is. That number is muscle effort, and it is the whole input.

Every EMG measurement uses three skin contacts. Two sit about 20 mm apart along the muscle and the sensor takes the difference between them. Room hum from wiring is the same on both and cancels; the muscle signal differs between them and survives. The third contact is the reference: an electrically quiet patch of skin that tells the amplifier what zero volts is on this body. The reference does not need to be far away and does not need to be on bone; it needs to be off the muscle being measured and not moving. Close is fine and cancels hum better.

Placement rules: pads over the belly of the muscle, about a third of the way down from the elbow, aligned along the arm. Firm, constant pressure; a loose band reads garbage. Damp-wipe the skin before wearing. Sweat helps. On a residual limb the muscles have moved, so the spots are found on Kyle with the sensors live, not from a chart, and marked on the skin the first week.

3. The sensor: DFRobot Gravity Analog EMG, SEN0240

Two of them. Each is two pieces on a short cable. The electrode module is a small flat pad with three metal dots on its face: the two measuring contacts and the reference between them, already spaced. It presses on the skin dry, with no gel and no stickers, and it sews into fabric. The signal board amplifies the microvolt signal about a thousand times, filters out hum and drift, and outputs it on a three-pin connector as a voltage centered near 1.5 V and swinging between 0 and 3 V with effort. The board plugs into the Pico with three wires: 3V3, GND, signal.

Why this sensor: it is the only hobby-grade EMG sensor built as a complete dry pad with the reference included, so each channel is one contact problem instead of three separate electrodes. It fits a fabric sleeve as-is, needs no consumables, and its output goes straight into the Pico's own analog pins. Two channels give open, close, rest, and co-contraction, which is everything the hand and the mouse need. An eight-channel raw front end (ADS1299) is the later upgrade for fine gestures and drops into the same sleeve, Pico, and pod.

4. Wiring

Pico pin	Connects to	Notes
3V3 OUT (pin 36)	Sensor 1 VCC, Sensor 2 VCC, BNO085 VIN	3.3 V so sensor output never exceeds the ADC range.
GND	Sensor 1 GND, Sensor 2 GND, BNO085 GND, button, motor driver	One star ground point in the pod.
GP26 (ADC0)	Sensor 1 signal (flexor)	
GP27 (ADC1)	Sensor 2 signal (extensor)	
GP28 (ADC2)	Battery divider midpoint	Battery+ -> 100k -> GP28 -> 100k -> GND. Reads half the battery voltage.
GP4 (I2C0 SDA)	BNO085 SDA	
GP5 (I2C0 SCL)	BNO085 SCL	
GP14	Tactile button to GND	Internal pull-up. Tap, 1 s hold, 2 s hold.
GP15	1k resistor -> 2N2222 base	Motor between 3V3 and collector, emitter to GND, 1N4148 across the motor, stripe toward 3V3.
VSYS / VBUS / GND	Pimoroni LiPo SHIM	Soldered under the Pico per the Pimoroni diagram. Battery plugs into the SHIM.

Sensor leads run along the sleeve to the pod, twisted, with about 4 inches of slack, ending in a JST-PH plug so either sensor unplugs. Leads route on the opposite side of the pod from the Pico's antenna.

5. The sleeve

A compression elbow sleeve in Kyle's size, snug enough to leave a faint mark after an hour and not tight enough to tingle. It starts just below the elbow crease and covers four to five inches of forearm, which reaches both muscle bellies and gives enough skin to grip so the sleeve cannot rotate. The two sensor pads are sewn to the inside face. A 2-inch elastic strap with hook-and-loop wraps over the pads on the outside so pressure is adjustable separately from the sleeve. The pod clips to the outer, back-of-arm side, away from the elbow crease, with a window cut for the USB port. A mark at the elbow end shows which way it goes on.

6. Firmware (CircuitPython on the Pico 2 W)

boot

```
--- read config: Wi-Fi list, Factum addr, hand addr, band id, thresholds, gains
--- start hardware watchdog
--- try home Wi-Fi and reach Factum for 5 s
|   reached      -> MODE 1 HAND-FACTUM
|   not reached  -> MODE 2 HAND-DIRECT: start hotspot "RoboKyle", wait for hand
--- last mode saved; button 1 s hold cycles HAND <-> MOUSE, buzz twice
```

sampling, every loop (~1 kHz)

```
--- read GP26, GP27; subtract 1.5 V center; take absolute value; accumulate
every 20 ms (50 frames/s)
--- effort per channel = average of the last 20 ms of absolute values
--- read BNO085 yaw, pitch, roll; read battery
--- MODE 1: send UDP frame to Factum
--- MODE 2: run thresholds + debounce on-board; on change POST open/close to hand;
|           keepalive POST to hand every 250 ms
--- feed watchdog
```

button held 2 s -> ESTOP: 5 UDP estop frames + 1 HTTP POST (Factum in mode 1, hand in mode 2), buzz 1 s
status server on port 80: GET /status, GET /battery, POST /buzz

Mode 3, MOUSE: Bluetooth on, Wi-Fi off. Advertises as a BLE HID mouse named RoboKyle Band, remembers the last host and reconnects on power up. Cursor: change in yaw and pitch every 10 ms times a gain, 1.5 degree dead zone, button tap recenters. Clutch: light flexor hold freezes the cursor. Left click: flexor burst over 80 ms, 250 ms refractory. Right click: extensor burst, same timing. Drag: flexor held over 400 ms. Double click: two flexor bursts within 350 ms. Scroll: extensor held plus arm tilt. Wi-Fi and Bluetooth share one radio, so HAND and MOUSE are exclusive.

7. Controls: what each mode does with the same three inputs

The band always produces the same three things: flexor effort, extensor effort, and arm orientation. Effort is a continuous number from rest to full squeeze, so it carries strength, not just on or off. What differs between modes is what the receiving side does with them. Grip menus and hand behaviour live in the hand adapter, never in the band's sensing code, so the same stream drives the arcade, a mouse, or anything else without changing the band.

Input	MODE 1 / 2: HAND	MODE 3: MOUSE	Arcade
Flexor effort	CLOSE. Strength of effort sets grip force (proportional, clamped).	Left click, drag, clutch	FIRE
Extensor effort	OPEN. Effort sets opening speed.	Right click, scroll modifier	Boost / brake
Co-contraction (both high)	Open the grip wheel. Held 2 s: ESTOP.	Toggle HAND / MOUSE (twice in 1 s)	Pause
Roll (forearm turn)	Wheel pointer while the wheel is open. Ignored otherwise.	Not used by default	Bank
Pitch (arm up/down)	Wheel pointer, second axis. Ignored otherwise.	Cursor Y / scroll rate	Climb / dive
Yaw (arm left/right)	Ignored.	Cursor X	Turn
Button	Tap: recenter. 1 s: mode. 2 s: ESTOP.	Same	Same

7.1 Grip selection: the wheel

The Brunel Hand ships with preset grips: fist, pinch, tripod, point, open palm, and others. Each grip lives at a fixed position on a clock face. Co-contraction opens the wheel; the arm points at a position; the physical hand previews that grip live, at zero force, as the arm moves; whatever the hand is showing when the wheel closes is the grip. No counting, no screen, no confirm gesture. The arcade draws the same wheel with the arm as the pointer, which is how he learns the positions.

states: REST, CLOSING, OPENING, WHEEL

REST (hand open, force below hold threshold)

co-contraction: flex AND ext above ON thresholds for > 150 ms

-> buzz once, enter WHEEL, save entry orientation as center

co-contraction while hand is gripping something -> ignored (open first)

WHEEL

angle = orientation relative to center, using roll and pitch as a 2D direction

sector = angle divided by (360 / N grips), N = 6 to 8; near center = no sector

on sector change:

buzz once

POST /grip/<name> to hand as a preview (posture, motors at low force)

at most one preview per 400 ms; only the current sector is ever sent

close when: arm returns near center, or no sector change for 1000 ms,

or 3 s total with no movement

-> long buzz, hand keeps the grip it is showing, return to REST

open burst -> cancel, hand returns to previous grip, return to REST

CLOSING (inside the selected grip)

force = clamp((flex_effort - flex_on) / (flex_max - flex_on), 0, 1) * force_limit

POST /close with force at most every 100 ms while it changes by > 5 %

Layout: rest and fist at the two easiest positions, straight ahead and arm rolled down, so the common grips need the least motion. Pinch, point, tripod, and open palm fill the remaining positions. Positions are fixed forever once chosen; changing them later costs relearning.

Haptic code, three signals only: one short buzz means the band registered a change (wheel opened, sector changed); one long buzz means the wheel closed and the grip is set; two short buzzes mean cancel or mode switch. ESTOP is a two-second buzz. No counts.

Why it holds up: entering the wheel needs a clean co-contraction, which does not happen by accident; the entry orientation resets the center every time so nothing drifts; roll and pitch inside the wheel are deliberate motions the arm rarely makes at rest; the hand only previews when it is empty, so a grip change can never drop what he is holding; and the preview is rate-limited so the hand never lags behind the arm.

8. What the band sends

One UDP packet every 20 milliseconds to Factum on port 5005:

```
{"id":"band1","seq":1234,"t":51230,"c":[0.12,0.81],"b":3.91,"o":[12.4,-3.1,0.8]}
```

id is the band's name. seq counts up by one each packet. t is milliseconds since boot. c is the two effort values in volts, flexor then extensor. b is battery voltage. o is yaw, pitch, roll in degrees.

Emergency stop, sent five times over UDP and once as an HTTP POST:

```
{"id":"band1","seq":1235,"t":52110,"estop":true}
```

What the band answers: GET /status returns id, battery, uptime, wifi signal, channels. GET /battery returns volts and percent. POST /buzz pulses the motor once.

What the hand needs to expose (Chad's side): an open endpoint, a close endpoint with force clamped, a grip endpoint that selects a preset by name at low force, a stop endpoint, an estop endpoint, and a watchdog that stops then opens if no keepalive arrives for 500 ms. Its Wi-Fi credentials are a list: home network first, the band's hotspot second.

What Factum does: ingests and logs every frame, computes rolling RMS, applies thresholds with hysteresis, debounces (3 frames to change, 5 to release), emits intent events, and drives adapters: the hand, the arcade over WebSocket, and later others. Calibration runs on Factum and writes the thresholds back into the band's config file, so HAND-DIRECT uses the same numbers.

9. Build order

Level	Build	Done when
1	Pico on USB, CircuitPython, Wi-Fi, UDP counter to a laptop listener	50 frames/s for 10 minutes, no gaps.
2	One SEN0240 on GP26, pad held on your own forearm	Fist raises the effort number clearly; rest drops it.
3	Second sensor on GP27, battery divider, button, motor, SHIM and LiPo	All read correctly; runs on battery.
4	Band firmware: frames, estop, status server, config file	Listener shows correct frames; estop and endpoints work.
5	Factum ingest, log, replay, thresholds, debounce, arcade adapter	Plane flies from your arm on robokyle.org.
6	Hand adapter, keepalive, estop, force clamp	Hand opens and closes from your arm under 300 ms; pulling the band's battery makes the hand stop then open.
7	HAND-DIRECT: hotspot, on-board thresholds	Hand works with the router unplugged.
8	Sleeve sewn, pads in, pod printed, cinch strap	Two hours worn, no dropouts, no skin redness.
9	Kyle: find spots with sensors live, calibrate, supervised week	Misfires under 1 per 10 minutes.
10	BNO085 and MOUSE mode	Cursor, click, drag, scroll, double click 20 of 20 on a TV and a laptop.

10. Safety

- The band never charges while worn. Nothing mains-powered touches the skin.
- Lost band for 500 ms: Factum sends stop, then open. Lost keepalive: the hand does the same on its own.
- Button held 2 s is ESTOP, straight to the hand, no classification in the way.
- Grip force is clamped in the adapter and starts low.
- Every stage is tested on the builder's arm before Kyle wears it.
- Every frame is logged from the first session.

11. Notes to self

- Measure Kyle's forearm at the widest point before ordering the sleeve. Buy the size that reads snug.
- Write the firmware in CircuitPython from the first line; the Bluetooth mouse library lives there.
- Build the arcade link before the hand link. Same intent stream, nothing to break, and it is the calibration screen.
- Ask Chad for four things: open, stop, keepalive, and grip-by-name at low force. Plus the credential list for the hotspot fallback.
- Damp wipe, snug band, pen marks on the skin for the first week.
- The Pico has three analog pins: two sensors and the battery. A third sensor needs an ADS1115 ADC on I2C.
- The wired fallback exists if Wi-Fi ever becomes the problem: Pico TX to Arduino RX, ground, a JST plug, text commands over serial. Not built until something forces it.
- Upgrade path: an ADS1299 breakout replaces the two sensor boards on the same Pico, sleeve, and pod, and a shared reference at the elbow end of the sleeve becomes correct at that point.

12. Parts list

#	Part	Qty	~\$	Purpose
1	DFRobot Gravity Analog EMG sensor, SEN0240	2	80	Dry three-dot pad plus signal board. One per muscle.
2	Raspberry Pi Pico 2 W	1	7	Brain, Wi-Fi, Bluetooth, analog inputs.
3	BNO085 9-axis IMU breakout	1	25	Arm orientation for the mouse and as context for the hand.
4	LiPo 1200 mAh 3.7 V, JST-PH	1	12	A day of use.
5	Pimoroni LiPo SHIM for Pico	1	10	Charging, power switch, battery to Pico.
6	Compression elbow sleeve, Kyle's size	1	10	The band.
7	Knit elastic 2 in wide, 1 yd	1	5	Cinch strap over the pads.
8	Hook-and-loop 2 in wide, 1 ft	1	4	Strap closure.
9	Silicone hookup wire 26-28 AWG, assorted	1	10	Sensor leads and pod wiring.
10	JST-PH 2.0 mm 3-pin connector kit	1	10	Unpluggable sensors; covers the wired fallback.
11	Heat shrink assortment	1	6	Lead strain relief.
12	Coin vibration motor 10 mm 3 V	1	2	Haptic click.
13	Electronics assortment: 2N2222, 1N4148, 1k, two 100k, 6 mm tactile button	1	10	Motor driver, battery divider, button.
14	M2 screw assortment	1	5	Pod.
	Total		~196	

Assumed on hand: soldering iron, solder, USB cable, 3D printer and filament, a computer for Factum. Prices are September 2026 averages from DFRobot, Adafruit, Pimoroni, and Amazon.